NAME: TANKISO MASOEBE

ADVANCED DATABASES WK3

1.SCENARIO ANALYSIS:

Optimization Scenario: Business Intelligence

A company wants to improve its decision-making by using business intelligence and optimization techniques. The company collects data about sales, customers, inventory, employee performance, and operating costs. However, managers find it difficult to determine which products should receive more attention and how much stock should be ordered. An optimization system can analyze historical and real-time data to identify the best decisions. For example, it can determine the quantity of products to order while minimizing storage costs and preventing shortages. It can also identify which products generate the highest profits and which sales periods have the greatest customer demand. By using dashboards, charts, and reports, managers can quickly understand the information and compare different business situations. The optimization process helps the company use its resources efficiently, reduce unnecessary costs, increase profits, and improve customer satisfaction. This scenario shows how business intelligence combined with optimization can support faster, more accurate, and data-driven business decisions.

2.CONCEPT RESEARCH:

Cost-based optimization is a technique used to choose the most efficient way of completing a task by comparing the estimated costs of different options. In databases, it is commonly used by query optimizers to determine the best execution plan for a SQL query. The optimizer considers factors such as processing time, memory usage, disk access, and the number of records involved. It then selects the plan with the lowest estimated cost. For example, it may choose an appropriate index instead of scanning an entire table. Cost-based optimization improves database performance by reducing resource usage, speeding up queries, and making better use of system resources.

3.TOOL PRACTICE:

```
-- Original query

SELECT *

FROM customers
```

WHERE city = 'Maseru';

-- Optimized query

CREATE INDEX idx_customers_city

ON customers(city);

SELECT customer_id, name, email

FROM customers

WHERE city = 'Maseru';

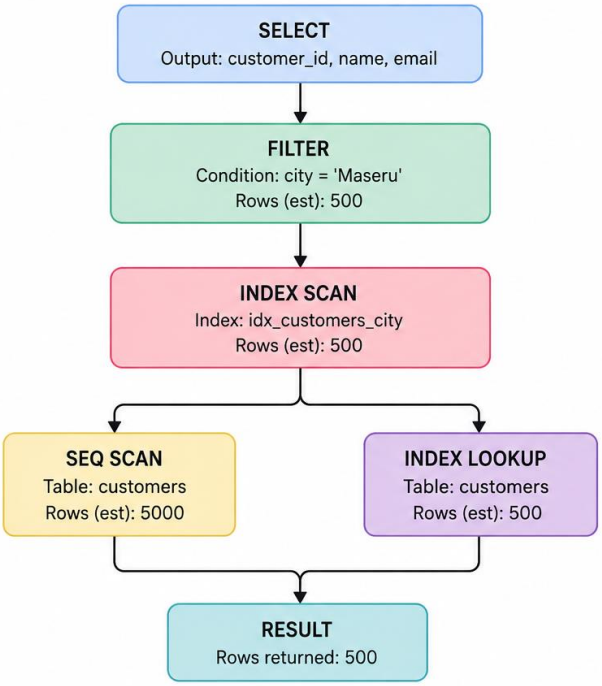
SQL Query

```
-- Original query
SELECT *
FROM customers
WHERE city = 'Maseru';

-- Optimized query
CREATE INDEX idx_customers_city
ON customers(city);

SELECT customer_id, name, email
FROM customers
WHERE city = 'Maseru';
```

sing PostgreSQL to optimize a SQL query helped me understand how database performance can be improved. The original query used SELECT *, which retrieved every column from the customers table, even when they were not needed. I optimized the query by selecting only the required columns and creating an index on the city column. The index allows PostgreSQL to find customers from Maseru more efficiently, especially when the table contains many records. This practical exercise showed me that optimization can reduce unnecessary data processing and improve query speed. I also learned that indexes should be used carefully because they require additional storage and can increase the cost of data updates.

4.DIAGRAM:

SQL QUERY PLAN DIAGRAM



EXPLANATION

This query plan shows how PostgreSQL executes a query to find customers from Maseru. First, the FILTER step applies the condition city = 'Maseru'.

The optimizer chooses to use the index idx_customers_city to quickly locate matching rows instead of scanning the whole table.

The INDEX SCAN finds the matching entries in the index. Then, INDEX LOOKUP retrieves the required rows from the customers table.

This plan is more efficient than a full SEQUENTIAL SCAN because it reads fewer rows, reduces disk I/O, and returns the result faster.